

Atividade de Pesquisa — PHP

Tema

Funções para Criptografia, Hash, Codificação e Proteção de Dados em PHP

Vinicius Moreira Silva

Lucas Humberto

Murilo Bertani

Lucas Marques

Objetivo

Pesquisar os principais recursos disponíveis na linguagem PHP para proteger informações, armazenar senhas com segurança, codificar dados e reduzir riscos de ataques em aplicações web. O objetivo desta atividade é compreender a importância da segurança da informação durante o desenvolvimento de sistemas, além de desenvolver habilidades de pesquisa, organização e documentação técnica.

1. Segurança em Aplicações Web

O que é Segurança da Informação?

Segurança da Informação (InfoSec) é a prática de proteger informações importantes contra acesso não autorizado, divulgação, uso, alteração ou interrupção ³. Ela abrange um conjunto de procedimentos e ferramentas que visam proteger dados corporativos confidenciais contra uso indevido, acesso não autorizado, interrupção ou destruição ⁵. A InfoSec garante que os dados organizacionais confidenciais estejam disponíveis para usuários autorizados, permaneçam confidenciais e mantenham sua integridade ³.

Por que proteger os dados dos usuários?

A proteção dos dados dos usuários é crucial por diversas razões. Primeiramente, a privacidade é considerada um direito humano fundamental em muitas jurisdições, e existem leis de proteção de dados para salvaguardar esse direito ⁴. Além disso, a proteção de dados ajuda a construir a confiança do cliente, pois as organizações demonstram que tratam as informações pessoais com cuidado ⁴.

Os dados pessoais podem ser usados indevidamente de várias maneiras se não forem mantidos em sigilo, como para fraudes, assédio, venda a anunciantes sem consentimento, ou para restringir a liberdade de expressão ⁴. Para as empresas, a falha na proteção de

dados pode resultar em multas regulatórias, penalidades legais e danos irreparáveis à reputação da marca 3 . Violações de dados podem levar à perda de clientes, roubo de propriedade intelectual e interrupção dos negócios 3 .

Quais são os principais riscos em aplicações desenvolvidas para a Internet?

Aplicações web estão constantemente expostas a uma variedade de ataques cibernéticos. Os riscos mais comuns incluem 1 2 5 :

- **Ataques de Injeção (Injection Attacks):** Ocorrem quando dados não confiáveis são enviados a um interpretador como parte de um comando ou consulta. Exemplos incluem SQL Injection (SQLi), onde um invasor explora vulnerabilidades na execução de consultas de banco de dados para obter acesso não autorizado, modificar ou destruir dados 1 2 .
- **Cross-Site Scripting (XSS):** Permite que um invasor injete scripts maliciosos do lado do cliente em uma página web, que são executados no navegador de outros usuários. Isso pode levar ao roubo de tokens de sessão, desfiguração da interface do usuário ou entrega de malware 1 2 .
- **Autenticação Quebrada (Broken Authentication):** Refere-se a falhas na forma como um site gerencia o login e as sessões, como políticas de senha fracas, armazenamento inseguro de senhas, IDs de sessão previsíveis ou falta de invalidação de sessões. Isso pode permitir que invasores se passem por usuários legítimos 2 .
- **Controle de Acesso Quebrado (Broken Access Control):** Falhas na implementação de restrições de acesso, permitindo que usuários acessem funcionalidades ou dados para os quais não têm permissão 2 .
- **Configurações de Segurança Incorretas (Security Misconfigurations):** Configurações padrão inseguras, permissões incorretas em arquivos e diretórios, ou recursos desnecessários habilitados que podem ser explorados por atacantes 2 .
- **Exposição de Dados Sensíveis (Sensitive Data Exposure):** Ocorre quando dados confidenciais não são protegidos adequadamente, seja em trânsito ou em repouso, tornando-os vulneráveis a roubo 2 .
- **Uso de Componentes com Vulnerabilidades Conhecidas (Using Components with Known Vulnerabilities):** A inclusão de bibliotecas, frameworks ou outros módulos de software com vulnerabilidades conhecidas pode abrir portas para ataques 2 .
- **Falsificação de Solicitação Entre Sites (Cross-Site Request Forgery - CSRF):** Induz uma vítima a fazer uma solicitação que utiliza sua autenticação ou autorização, permitindo que um invasor execute ações como se fosse o usuário 1 2 .

- **Ataques de Negação de Serviço (DoS/DDoS):** Visam sobrecarregar o servidor ou a infraestrutura de uma aplicação com tráfego malicioso, tornando-a indisponível para usuários legítimos 1 .
- **Violação de APIs (API Breaches):** APIs, que permitem a comunicação entre aplicações, podem ter vulnerabilidades que permitem a invasores enviar códigos maliciosos ou interceptar dados confidenciais 1 .

Essas vulnerabilidades representam uma superfície de ataque crescente e complexa, exigindo que as organizações implementem medidas de segurança robustas para proteger suas aplicações e os dados de seus usuários 2 .

Referências

- [1] Cloudflare. O que é segurança de aplicativos web? Disponível em: <https://www.cloudflare.com/pt-br/learning/security/what-is-web-application-security/>. Acesso em: 5 jul. 2026.
- [2] Aikido Security. Top 10 Web Application Security Vulnerabilities Every Team Should Know. Disponível em: <https://pt.aikido.dev/blog/top-web-application-security-vulnerabilities>. Acesso em: 5 jul. 2026.
- [3] IBM. O que é segurança da informação (InfoSec)? Disponível em: <https://www.ibm.com/br-pt/think/topics/information-security>. Acesso em: 5 jul. 2026.
- [4] Cloudflare. O que é privacidade de dados? Disponível em: <https://www.cloudflare.com/pt-br/learning/privacy/what-is-data-privacy/>. Acesso em: 5 jul. 2026.
- [5] Microsoft. O que é InfoSec (Segurança da informação)? Disponível em: <https://www.microsoft.com/pt-br/security/business/security-101/what-is-information-security-infosec>. Acesso em: 5 jul. 2026.

2. Criptografia, Hash e Codificação

Embora frequentemente usados de forma intercambiável, **Criptografia**, **Hash** e **Codificação (Encoding)** são conceitos distintos com propósitos e mecanismos diferentes na segurança da informação 1 2 3 .

Criptografia

O que é: A criptografia é a ciência que estuda técnicas para transformar informações de forma a torná-las secretas e ilegíveis para pessoas não autorizadas. Seu objetivo principal é garantir a **confidencialidade** dos dados, ou seja, que apenas os destinatários legítimos possam acessá-los e compreendê-los 1 2 .

Como funciona: A criptografia utiliza algoritmos e chaves (secretas) para embaralhar os dados (texto simples) em um formato ilegível (texto cifrado). Para reverter o processo e obter o texto original, é necessária a chave correta e o algoritmo de descryptografia 1 2 .

Existem dois tipos principais de criptografia:

- **Criptografia Simétrica:** Utiliza uma única chave secreta para criptografar e descryptografar os dados. Exemplos incluem AES, DES, 3DES e Blowfish. É eficiente para grandes volumes de dados, mas a distribuição segura da chave é um desafio 1 .
- **Criptografia Assimétrica (Chave Pública):** Utiliza um par de chaves – uma chave pública para criptografar e uma chave privada para descryptografar. A chave pública pode ser amplamente distribuída, enquanto a chave privada é mantida em segredo. Exemplos incluem RSA e ECC. É usada para garantir confidencialidade, autenticidade e não repúdio, sendo fundamental em assinaturas digitais e comunicação segura na internet (HTTPS) 1 .

Exemplos de utilização:

- **Comunicação segura:** Proteção de e-mails, mensagens instantâneas e tráfego web (HTTPS) 1 .
- **Armazenamento de dados:** Criptografia de discos rígidos, bancos de dados e arquivos em nuvem 1 .
- **Assinaturas digitais:** Verificação da autenticidade e integridade de documentos e softwares 1 .

Hash

O que é: Uma função de hash (ou algoritmo de hash) é um processo unidirecional que transforma uma entrada de dados (de qualquer tamanho) em uma sequência de caracteres de tamanho fixo, chamada de valor hash, resumo criptográfico ou *digest* 1 2 . O objetivo principal do hash é garantir a **integridade** dos dados, ou seja, verificar se os dados não foram alterados 2 .

Como funciona: As funções de hash são projetadas para serem determinísticas (a mesma entrada sempre produz a mesma saída), rápidas de calcular, e, crucialmente, impossíveis de reverter (não é possível obter a entrada original a partir do hash). Além disso, uma pequena alteração na entrada deve resultar em um hash completamente diferente (efeito avalanche), e a probabilidade de duas entradas diferentes produzirem o mesmo hash (colisão) deve ser extremamente baixa 2 3 .

Exemplos de algoritmos: SHA-1 (considerado obsoleto para segurança), SHA-256, SHA-3 e MD5 (obsoleto) 2 3 .

Exemplos de utilização:

- **Verificação de integridade de arquivos:** Comparar o hash de um arquivo baixado com o hash fornecido pelo desenvolvedor para garantir que o arquivo não foi corrompido ou adulterado 2.
- **Armazenamento seguro de senhas:** Em vez de armazenar senhas em texto puro, armazena-se o hash da senha. Quando um usuário tenta fazer login, a senha fornecida é "hasheada" e comparada com o hash armazenado. Se os hashes coincidirem, a senha está correta 2.
- **Assinaturas digitais:** O hash de um documento é assinado digitalmente, garantindo a integridade do documento 1.

Codificação (Encoding)

O que é: A codificação é o processo de transformar dados de um formato para outro, usando um esquema publicamente disponível. Seu propósito **não é** manter a informação em segredo, mas sim garantir que os dados possam ser consumidos corretamente por diferentes sistemas ou para otimizar seu armazenamento e transmissão 1 2.

Como funciona: A codificação utiliza algoritmos conhecidos para converter dados em um formato diferente, que pode ser facilmente revertido (decodificado) usando o mesmo algoritmo, sem a necessidade de uma chave secreta 1 2.

Exemplos de algoritmos: Base64, UTF-8, ASCII, codificação de URL 1 2.

Exemplos de utilização:

- **Transferência de dados:** Converter dados binários para um formato de texto (como Base64) para que possam ser transmitidos via e-mail ou incluídos em URLs 1 2.
- **Compatibilidade de caracteres:** UTF-8 e ASCII são usados para representar caracteres de texto em diferentes sistemas, garantindo que sejam exibidos corretamente 1.
- **Otimização:** Reduzir o tamanho de arquivos para armazenamento ou transmissão 1.

Diferenças Chave

Característica	Criptografia	Hash	Codificação (Encoding)
Propósito	Confidencialidade (manter segredo)	Integridade (verificar não alteração)	Usabilidade/Compatibilidade (mudar formato)
Reversibilidade	Reversível com a chave correta	Irreversível (unidirecional)	Reversível com o algoritmo conhecido

Chave Secreta	Sim (simétrica ou assimétrica)	Não	Não
Exemplos	AES, RSA, HTTPS	SHA-256, MD5 (obsoleto), password_hash()	Base64, UTF-8, ASCII
Segurança	Alta (se bem implementada)	Garante integridade, não confidencialidade	Nenhuma (não visa segurança)

Referências (Continuação)

[6] Stack Overflow. Qual a diferença entre codificação, criptografia e cálculo hash? Disponível em: <https://pt.stackoverflow.com/questions/162369/qual-a-diferen%C3%A7a-entre-codifica%C3%A7%C3%A3o-criptografia-e-c%C3%A1lculo-hash>. Acesso em: 5 jul. 2026.

[7] DEV Community. Encoding, Hashing e Encryption: Qual a diferença? Disponível em: <https://dev.to/kotlinautas/encoding-hashing-e-encryption-qual-a-diferenca-29gg>. Acesso em: 5 jul. 2026.

[8] Ti Para Leigo. Hashing x criptografia x codificação (Encoding) x ofuscação (Obfuscation). Disponível em: <https://tiparaleigo.wordpress.com/2022/01/14/hashing-x-criptografia-x-codificacao-encoding-x-ofuscacao-obfuscation/>. Acesso em: 5 jul. 2026.

3. Funções de Hash no PHP

O PHP oferece uma API robusta para manipulação de hashes, essencial para a segurança de senhas e verificação de integridade ⁹ ¹³.

password_hash()

O que é: É a função recomendada para criar hashes de senhas de forma segura ⁹. Ela utiliza algoritmos fortes e gerencia automaticamente a criação de um *salt* único para cada senha ¹³.

Para que serve: Serve para transformar uma senha em texto puro em um hash seguro que pode ser armazenado no banco de dados ¹³.

Exemplo: `password_hash("minha_senha", PASSWORD_DEFAULT);`

password_verify()

O que é: Função usada para verificar se uma senha fornecida pelo usuário corresponde a um hash armazenado anteriormente ¹⁰.

Para que serve: Serve para autenticar usuários sem nunca precisar descriptografar o hash ou armazenar a senha original 10.

Exemplo: `if (password_verify($senha_digitada, $hash_armazenado)) { // Sucesso }`

hash()

O que é: Uma função genérica para gerar hashes de dados usando diversos algoritmos (como SHA-256) 13.

Para que serve: Diferente das funções específicas de senha, o `hash()` é usado para verificar a integridade de arquivos ou dados genéricos 13.

Quando devem ser utilizadas?

- **password_hash() e password_verify():** Devem ser utilizadas sempre que houver necessidade de armazenar e validar senhas de usuários 9.
- **hash():** Deve ser utilizada para gerar resumos de arquivos, verificar se dados foram alterados durante a transmissão ou para criar identificadores únicos baseados em conteúdo 13.

Algoritmos Recomendados Atualmente

Atualmente, o algoritmo recomendado pelo PHP (através da constante `PASSWORD_DEFAULT`) é o **bcrypt** 13. No entanto, o **Argon2** (Argon2i ou Argon2id) é considerado o estado da arte e é altamente recomendado se o sistema tiver suporte, pois oferece maior resistência a ataques de força bruta e hardware especializado (GPUs/ASICs) 13.

4. Funções de Codificação

No PHP, as funções de codificação são usadas para transformar dados para transporte ou compatibilidade, não para segurança 11.

base64_encode() e base64_decode()

Para que servem: Servem para converter dados binários em uma representação de texto (MIME base64) e vice-versa 11. Isso garante que os dados sobrevivam ao transporte em camadas que não são "limpas" para 8 bits, como corpos de e-mail 15.

Situações de uso:

- Envio de anexos em e-mails 15.
- Inclusão de pequenas imagens diretamente no HTML/CSS (Data URIs) 15.

- Passagem de dados complexos em URLs (usando variantes seguras para URL) 15.

Por que Base64 não é criptografia: Porque a Base64 não utiliza chaves e o seu algoritmo é público e facilmente reversível por qualquer pessoa 7. O objetivo é a **codificação**, não a **confidencialidade** 1.

5. Criptografia no PHP (OpenSSL)

Para criptografia real (confidencialidade), o PHP utiliza a extensão **OpenSSL** 14.

O que é OpenSSL?

O OpenSSL é uma biblioteca de software robusta, comercial e de código aberto que implementa os protocolos SSL e TLS, além de uma biblioteca de criptografia de propósito geral 14.

Para que serve?

No PHP, serve para realizar criptografia simétrica e assimétrica, gerenciar certificados digitais e gerar números aleatórios criptograficamente seguros 14.

Funções Principais

- **openssl_encrypt():** Criptografa dados usando um algoritmo (como AES-256-GCM) e uma chave 14.
 - **openssl_decrypt():** Descriptografa dados que foram protegidos com `openssl_encrypt()`, exigindo a mesma chave e o IV (Vetor de Inicialização) correto 14.
 - **openssl_random_pseudo_bytes():** Gera uma sequência de bytes aleatórios, fundamental para criar chaves e IVs seguros 14.
-

Referências (Continuação)

[9] PHP Manual. `password_hash`. Disponível em:

<https://www.php.net/manual/en/function.password-hash.php>. Acesso em: 5 jul. 2026.

[10] PHP Manual. `password_verify`. Disponível em:

<https://www.php.net/manual/en/function.password-verify.php>. Acesso em: 5 jul. 2026.

[11] PHP Manual. `base64_encode`. Disponível em:

<https://www.php.net/manual/en/function.base64-encode.php>. Acesso em: 5 jul. 2026.

[12] OWASP. SQL Injection. Disponível em: https://owasp.org/www-community/attacks/SQL_Injection. Acesso em: 5 jul. 2026.

[13] SitePoint. Hashing Passwords with the PHP 5.5 Password Hashing API. Disponível em: <https://www.sitepoint.com/hashing-passwords-php-5-5-password-hashing-api/>. Acesso em: 5 jul. 2026.

[14] PHP Manual. openssl_encrypt. Disponível em: <https://www.php.net/manual/en/function.openssl-encrypt.php>. Acesso em: 5 jul. 2026.

[15] Dev.to. Php Base64 encode/decode – best practices and use cases. Disponível em: <https://dev.to/inspector/php-base64-encodedecode-best-practices-and-use-cases-4mdm>. Acesso em: 5 jul. 2026.

6. Proteção de Senhas

A proteção de senhas é um dos aspectos mais críticos da segurança em qualquer aplicação web 9 13.

Como uma senha deve ser armazenada corretamente?

Uma senha **nunca** deve ser armazenada em seu formato original (texto puro). Ela deve ser processada por um algoritmo de hash criptográfico forte e lento (como bcrypt ou Argon2) antes de ser salva no banco de dados 9 . O uso de funções como `password_hash()` no PHP automatiza esse processo corretamente 13.

Por que nunca devemos salvar senhas em texto puro?

Se um invasor obtiver acesso ao banco de dados (por meio de uma injeção de SQL, por exemplo), ele terá acesso imediato a todas as senhas de todos os usuários 12 . Como muitos usuários reutilizam senhas em diferentes serviços, um vazamento em um sistema pode comprometer contas em outros sites (e-mails, redes sociais, bancos) 3 .

O que é Salt?

O *salt* (sal) é uma sequência aleatória de dados adicionada à senha antes de ser aplicada a função de hash 13.

- **Para que serve:** Garante que, mesmo que dois usuários tenham a mesma senha, seus hashes resultantes sejam diferentes 13.
- **Importância:** Impede ataques de *Rainbow Tables* (tabelas de hashes pré-calculados) e dificulta ataques de força bruta em larga escala 13.

O que torna um algoritmo de hash seguro?

Um algoritmo de hash seguro para senhas deve ser:

1. **Unidirecional:** Impossível de reverter 2 .

- 2. **Resistente a colisões:** Difícil de encontrar duas entradas que gerem o mesmo hash 2 .
 - 3. **Lento (Custo Computacional):** Deve exigir tempo e recursos computacionais para ser calculado, tornando ataques de força bruta inviáveis economicamente 13 .
-

7. Proteção contra Ataques

O PHP oferece diversas ferramentas e extensões para mitigar ataques comuns 12 .

SQL Injection (SQLi)

- **O que é:** Inserção de comandos SQL maliciosos em campos de entrada para manipular o banco de dados 12 .
- **Prevenção:** Utilizar **Prepared Statements** (Consultas Preparadas) com PDO ou MySQLi. Isso separa o comando SQL dos dados do usuário, impedindo que a entrada seja interpretada como código 12 .

Cross-Site Scripting (XSS)

- **O que é:** Injeção de scripts maliciosos (JS) em páginas visualizadas por outros usuários 1 2 .
- **Prevenção:** Escapar toda saída de dados usando `htmlspecialchars()` ou `strip_tags()` antes de exibir conteúdo gerado por usuários no navegador 2 .

Cross-Site Request Forgery (CSRF)

- **O que é:** Induzir um usuário autenticado a executar ações indesejadas em uma aplicação na qual ele está logado 1 .
 - **Prevenção:** Utilizar **Tokens CSRF** (um valor aleatório único gerado para cada sessão/formulário) que deve ser validado pelo servidor em cada requisição de alteração de dados 1 .
-

8. Aplicações Práticas

As técnicas de segurança pesquisadas são fundamentais em diversos sistemas reais:

- **Sistemas de Login:** Uso de `password_hash()` e `password_verify()` para autenticação segura.
- **Comércio Eletrônico e Internet Banking:** Uso de OpenSSL (HTTPS e criptografia de dados sensíveis) para proteger transações financeiras.

- **Redes Sociais:** Proteção contra XSS e CSRF para evitar sequestro de contas e postagens não autorizadas.
- **Sistemas Escolares:** Controle de acesso e integridade de dados para garantir que apenas pessoas autorizadas alterem notas e registros.

Importância: Essas técnicas protegem a privacidade dos usuários, garantem a continuidade dos negócios e evitam perdas financeiras e danos à reputação das instituições.

9. Boas Práticas de Segurança

Para desenvolver aplicações PHP seguras, recomenda-se:

- **Validar e filtrar todas as entradas do usuário:** Nunca confiar em dados vindos de formulários ou URLs.
 - **Utilizar Prepared Statements:** Sempre usar PDO para interações com o banco de dados.
 - **Proteger sessões:** Usar `session_regenerate_id()` e cookies seguros (`HttpOnly` , `Secure`).
 - **Utilizar HTTPS:** Criptografar todo o tráfego entre o cliente e o servidor.
 - **Manter o PHP atualizado:** Versões antigas podem conter vulnerabilidades conhecidas.
 - **Configurações Seguras:** Desabilitar a exibição de erros em produção e proteger arquivos `.env` ou de configuração.
-

10. Conclusão

A segurança em aplicações web é um pilar fundamental do desenvolvimento moderno. Ela não é apenas uma característica técnica, mas um compromisso com a proteção da privacidade e dos ativos dos usuários. Entre os recursos pesquisados, o **hashing de senhas** com algoritmos modernos como Argon2 destaca-se como o mais vital para evitar o comprometimento massivo de contas. Através desta pesquisa, compreendemos que a segurança deve ser pensada em camadas, combinando codificação correta, criptografia forte e práticas de desenvolvimento defensivo para mitigar riscos em um ambiente digital cada vez mais hostil.

Referências Finais (ABNT)

AIKIDO SECURITY. **Top 10 Web Application Security Vulnerabilities Every Team Should Know**. Disponível em: <https://pt.aikido.dev/blog/top-web-application-security->

[vulnerabilities](#). Acesso em: 05 jul. 2026.

CLOUDFLARE. **O que é segurança de aplicativos web?** Disponível em:

<https://www.cloudflare.com/pt-br/learning/security/what-is-web-application-security/>.

Acesso em: 05 jul. 2026.

CLOUDFLARE. **O que é privacidade de dados?** Disponível em:

<https://www.cloudflare.com/pt-br/learning/privacy/what-is-data-privacy/>. Acesso em: 05 jul. 2026.

DEV COMMUNITY. **Encoding, Hashing e Encryption: Qual a diferença?** Disponível em:

<https://dev.to/kotlinautas/encoding-hashing-e-encryption-qual-a-diferenca-29gg>. Acesso em: 05 jul. 2026.

IBM. **O que é segurança da informação (InfoSec)?** Disponível em:

<https://www.ibm.com/br-pt/think/topics/information-security>. Acesso em: 05 jul. 2026.

MICROSOFT. **O que é InfoSec (Segurança da informação)?** Disponível em:

<https://www.microsoft.com/pt-br/security/business/security-101/what-is-information-security-infosec>. Acesso em: 05 jul. 2026.

OWASP. **SQL Injection**. Disponível em: https://owasp.org/www-community/attacks/SQL_Injection. Acesso em: 05 jul. 2026.

PHP. **Password Hashing**. Disponível em:

<https://www.php.net/manual/en/book.password.php>. Acesso em: 05 jul. 2026.

PHP. **OpenSSL**. Disponível em: <https://www.php.net/manual/en/book.openssl.php>. Acesso em: 05 jul. 2026.

STACK OVERFLOW. **Qual a diferença entre codificação, criptografia e cálculo hash?**

Disponível em: <https://pt.stackoverflow.com/questions/162369/qual-a-diferen%C3%A7a-entre-codifica%C3%A7%C3%A3o-criptografia-e-c%C3%A1lculo-hash>. Acesso em: 05 jul. 2026.